AMS 580 Quiz 4

Kachun Huang

2025-04-07

AMS 580 Quiz 4

Import Library

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3

## Loading required package: ggplot2

## Loading required package: lattice

## Warning: package 'lattice' was built under R version 4.3.3
```

```
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.3.3

## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.
```

```
##
## Attaching package: 'randomForest'

## The following object is masked from 'package:ggplot2':
##
##     margin

## The following object is masked from 'package:dplyr':
##
##     combine
```

```
library(tidyverse)
```

```
## Warning: package 'purrr' was built under R version 4.3.3

## Warning: package 'lubridate' was built under R version 4.3.3

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.0     v stringr   1.5.1
## v lubridate 1.9.4     v tibble    3.2.1
## v purrr     1.0.4     v tidyr     1.3.1
## v readr     2.1.5

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x randomForest::combine() masks dplyr::combine()
## x dplyr::filter()         masks stats::filter()
## x dplyr::lag()            masks stats::lag()
## x purrr::lift()           masks caret::lift()
## x randomForest::margin()  masks ggplot2::margin()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(rpart)
```

```
## Warning: package 'rpart' was built under R version 4.3.3
```

```
library(rpart.plot)
```

```
## Warning: package 'rpart.plot' was built under R version 4.3.3
```

```
library(rattle)
```

```
## Loading required package: bitops

## Warning: package 'bitops' was built under R version 4.3.3

## Rattle: A free graphical interface for data science with R.
## Version 5.5.1 Copyright (c) 2006-2021 Togaware Pty Ltd.
## Type 'rattle()' to shake, rattle, and roll your data.
##
## Attaching package: 'rattle'
##
## The following object is masked from 'package:randomForest':
##
##     importance
```

```r
library(caTools)
```

```
## Warning: package 'caTools' was built under R version 4.3.3
```

Regression Task

Import Data

```r
house <- read.csv("/Users/thomashuang/Downloads/Ames_Housing_Data.csv")
head(house)
```

```
##   Id LotArea OverallQual OverallCond YearBuilt YearRemodAdd CentralAir
## 1  1    8450           7           5      2003         2003          Y
## 2  2    9600           6           8      1976         1976          Y
## 3  3   11250           7           5      2001         2002          Y
## 4  4    9550           7           5      1915         1970          Y
## 5  5   14260           8           5      2000         2000          Y
## 6  6   14115           5           5      1993         1995          Y
##   X1stFlrSF X2ndFlrSF GrLivArea FullBath HalfBath BedroomAbvGr KitchenAbvGr
## 1       856       854      1710        2        1            3            1
## 2      1262         0      1262        2        0            3            1
## 3       920       866      1786        2        1            3            1
## 4       961       756      1717        1        0            3            1
## 5      1145      1053      2198        2        1            4            1
## 6       796       566      1362        1        1            1            1
##   TotRmsAbvGrd Fireplaces GarageCars GarageArea YrSold SalePrice
## 1            8          0          2        548   2008    208500
## 2            6          1          2        460   2007    181500
## 3            6          1          2        608   2008    223500
## 4            7          1          3        642   2006    140000
## 5            9          1          3        836   2008    250000
## 6            5          0          2        480   2009    143000
```

```r
str(house)
```

```
## 'data.frame':    1460 obs. of  20 variables:
##  $ Id          : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ LotArea     : int  8450 9600 11250 9550 14260 14115 10084 10382 6120 7420 ...
##  $ OverallQual : int  7 6 7 7 8 5 8 7 7 5 ...
##  $ OverallCond : int  5 8 5 5 5 5 5 6 5 6 ...
##  $ YearBuilt   : int  2003 1976 2001 1915 2000 1993 2004 1973 1931 1939 ...
##  $ YearRemodAdd: int  2003 1976 2002 1970 2000 1995 2005 1973 1950 1950 ...
##  $ CentralAir  : chr  "Y" "Y" "Y" "Y" ...
##  $ X1stFlrSF   : int  856 1262 920 961 1145 796 1694 1107 1022 1077 ...
##  $ X2ndFlrSF   : int  854 0 866 756 1053 566 0 983 752 0 ...
##  $ GrLivArea   : int  1710 1262 1786 1717 2198 1362 1694 2090 1774 1077 ...
##  $ FullBath    : int  2 2 2 1 2 1 2 2 2 1 ...
##  $ HalfBath    : int  1 0 1 0 1 1 0 1 0 0 ...
##  $ BedroomAbvGr: int  3 3 3 3 4 1 3 3 2 2 ...
```

```
##  $ KitchenAbvGr: int  1 1 1 1 1 1 1 1 2 2 ...
##  $ TotRmsAbvGrd: int  8 6 6 7 9 5 7 7 8 5 ...
##  $ Fireplaces  : int  0 1 1 1 1 0 1 2 2 2 ...
##  $ GarageCars  : int  2 2 2 3 3 2 2 2 2 1 ...
##  $ GarageArea  : int  548 460 608 642 836 480 636 484 468 205 ...
##  $ YrSold      : int  2008 2007 2008 2006 2008 2009 2007 2009 2008 2008 ...
##  $ SalePrice   : int  208500 181500 223500 140000 250000 143000 307000 200000 129900 118000 ...
```

Remove Missing Values

```
house <- na.omit(house)
str(house)
```

```
## 'data.frame':    1460 obs. of  20 variables:
##  $ Id          : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ LotArea     : int  8450 9600 11250 9550 14260 14115 10084 10382 6120 7420 ...
##  $ OverallQual : int  7 6 7 7 8 5 8 7 7 5 ...
##  $ OverallCond : int  5 8 5 5 5 5 5 6 5 6 ...
##  $ YearBuilt   : int  2003 1976 2001 1915 2000 1993 2004 1973 1931 1939 ...
##  $ YearRemodAdd: int  2003 1976 2002 1970 2000 1995 2005 1973 1950 1950 ...
##  $ CentralAir  : chr  "Y" "Y" "Y" "Y" ...
##  $ X1stFlrSF   : int  856 1262 920 961 1145 796 1694 1107 1022 1077 ...
##  $ X2ndFlrSF   : int  854 0 866 756 1053 566 0 983 752 0 ...
##  $ GrLivArea   : int  1710 1262 1786 1717 2198 1362 1694 2090 1774 1077 ...
##  $ FullBath    : int  2 2 2 1 2 1 2 2 2 1 ...
##  $ HalfBath    : int  1 0 1 0 1 1 0 1 0 0 ...
##  $ BedroomAbvGr: int  3 3 3 3 4 1 3 3 2 2 ...
##  $ KitchenAbvGr: int  1 1 1 1 1 1 1 1 2 2 ...
##  $ TotRmsAbvGrd: int  8 6 6 7 9 5 7 7 8 5 ...
##  $ Fireplaces  : int  0 1 1 1 1 0 1 2 2 2 ...
##  $ GarageCars  : int  2 2 2 3 3 2 2 2 2 1 ...
##  $ GarageArea  : int  548 460 608 642 836 480 636 484 468 205 ...
##  $ YrSold      : int  2008 2007 2008 2006 2008 2009 2007 2009 2008 2008 ...
##  $ SalePrice   : int  208500 181500 223500 140000 250000 143000 307000 200000 129900 118000 ...
```

After removing the missing value, there are 1460 obs left.

Data Splitting

```
set.seed(123)
training.samples <- house$SalePrice %>% createDataPartition(p=0.75, list=FALSE)
train.data <- house[training.samples,]
test.data <- house[-training.samples,]
```

Prune CART

```r
set.seed(123)

model_CART <- train(
  SalePrice ~.,
  data = train.data,
  method = "rpart",
  trControl = trainControl("cv", number = 10)
)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

Access the best tune

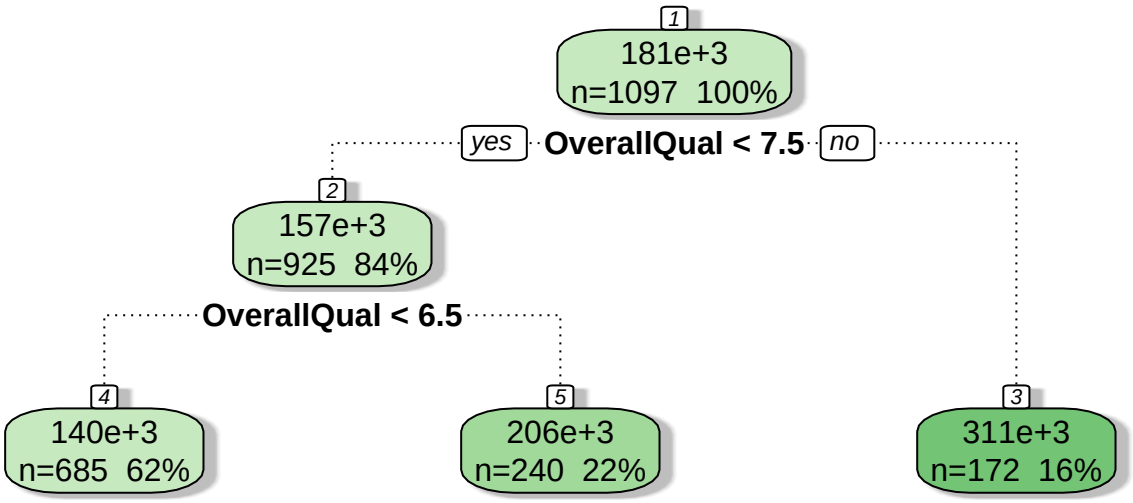
```r
model_CART$bestTune
```

```
##           cp
## 1 0.07317336
```

The best CP value is 0.07317

Plot the best model

```r
fancyRpartPlot(model_CART$finalModel)
```



Rattle 2025−Apr−07 18:05:09 thomashuang

Prediction

```
pred_CART <- predict(
  model_CART,
  test.data
)
```

Test RMSE

```
RMSE(pred_CART,test.data$SalePrice)
```

```
## [1] 50958.41
```

Best Random Forest

```
set.seed(123)

# 10-fold CV
control <- trainControl(
  method = "cv",
  number = 10
  )

# train the rf using 10-fold CV
model_rf <- train(
  SalePrice~.,
  data = train.data,
  method = "rf",
  trControl = control
  )

print(model_rf)
```

```
## Random Forest
##
## 1097 samples
##   19 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 987, 988, 986, 987, 988, 989, ...
## Resampling results across tuning parameters:
##
##   mtry  RMSE      Rsquared   MAE
##    2    28397.09  0.8909575  18508.34
##   10    26651.02  0.8953625  17832.16
##   19    27725.30  0.8848516  18560.06
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was mtry = 10.
```

Access bestTune

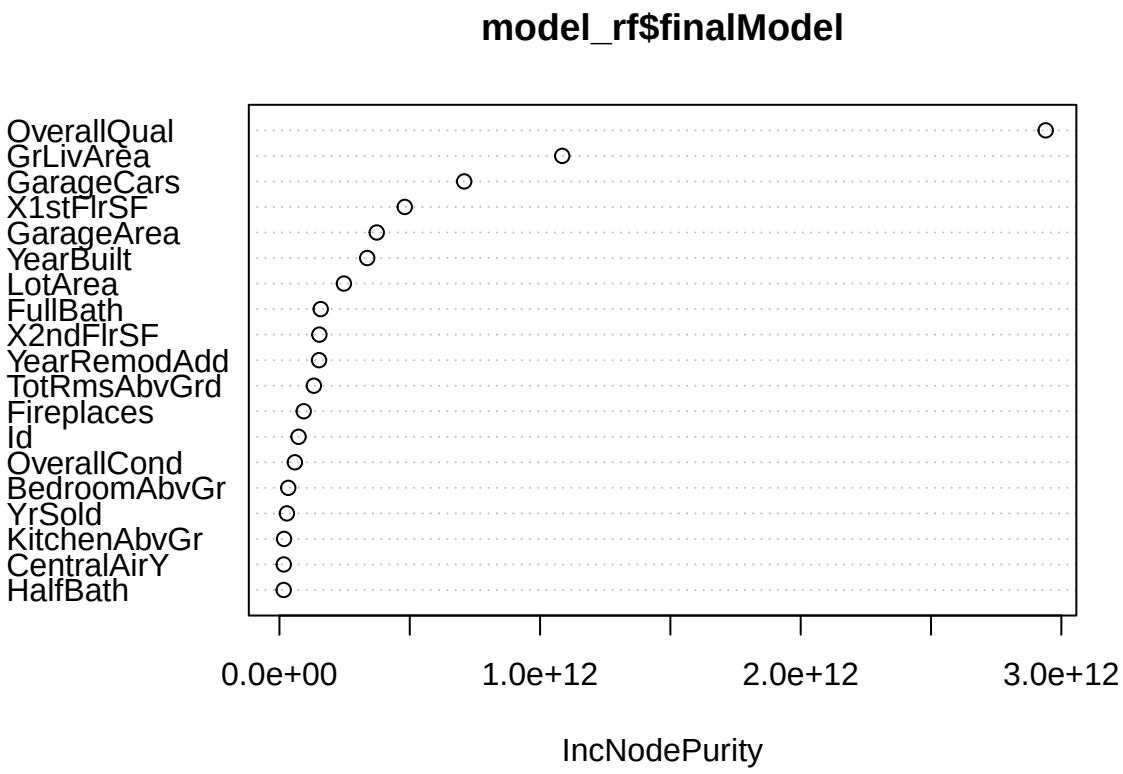
```
print(model_rf$bestTune)
```

```
##   mtry
## 2   10
```

The best mtry value is 10

Plot importance measures

```
varImpPlot(model_rf$finalModel)
```



Prediction

```
pred_rf <- predict(
  model_rf,
  newdata = test.data
)
```

Test RMSE

```
RMSE(pred_rf, test.data$SalePrice)
```

```
## [1] 37341
```

Which is better?

The Test RMSE of CART is 50958.41

The Test RMSE of Random Forest is 37341

In terms of test RMSE, random forest performs better than CART.

The possible reasons of random forest performs better than CART are

- CART generally overfit on training data, result in high rmse when evaluate with test data
- Random Forest performs better because of its algorithm
  - by aggregating all the prediction of all tree created from bootstrap sample
  - It allows the model to foresee unseen data, so lower the rmse when evaluate with test data

Classification Task

Import Data

```
titanic <- read.csv("/Users/thomashuang/Downloads/Titanic.csv")
```

```
head(titanic)
```

```
##   PassengerId Survived Pclass
## 1           1        0      3
## 2           2        1      1
## 3           3        1      3
## 4           4        1      1
## 5           5        0      3
## 6           6        0      3
##                                                 Name    Sex Age SibSp Parch
## 1                            Braund, Mr. Owen Harris   male  22     1     0
## 2 Cumings, Mrs. John Bradley (Florence Briggs Thayer) female  38     1     0
## 3                             Heikkinen, Miss. Laina female  26     0     0
## 4       Futrelle, Mrs. Jacques Heath (Lily May Peel) female  35     1     0
## 5                           Allen, Mr. William Henry   male  35     0     0
## 6                                   Moran, Mr. James   male  NA     0     0
##             Ticket    Fare Cabin Embarked
## 1        A/5 21171  7.2500              S
## 2         PC 17599 71.2833   C85        C
## 3 STON/O2. 3101282  7.9250              S
## 4           113803 53.1000  C123        S
## 5           373450  8.0500              S
## 6           330877  8.4583              Q
```

```r
str(titanic)
```

```
## 'data.frame':    891 obs. of  12 variables:
##  $ PassengerId: int  1 2 3 4 5 6 7 8 9 10 ...
##  $ Survived   : int  0 1 1 1 0 0 0 0 1 1 ...
##  $ Pclass     : int  3 1 3 1 3 3 1 3 3 2 ...
##  $ Name       : chr  "Braund, Mr. Owen Harris" "Cumings, Mrs. John Bradley (Florence Briggs Thayer)"
##  $ Sex        : chr  "male" "female" "female" "female" ...
##  $ Age        : num  22 38 26 35 35 NA 54 2 27 14 ...
##  $ SibSp      : int  1 1 0 1 0 0 0 3 0 1 ...
##  $ Parch      : int  0 0 0 0 0 0 0 1 2 0 ...
##  $ Ticket     : chr  "A/5 21171" "PC 17599" "STON/O2. 3101282" "113803" ...
##  $ Fare       : num  7.25 71.28 7.92 53.1 8.05 ...
##  $ Cabin      : chr  "" "C85" "" "C123" ...
##  $ Embarked   : chr  "S" "C" "S" "S" ...
```

Exclude some columns

```r
titanic <- subset(titanic, select = -c(PassengerId, Name, Ticket, Cabin))
```

```r
str(titanic)
```

```
## 'data.frame':    891 obs. of  8 variables:
##  $ Survived: int  0 1 1 1 0 0 0 0 1 1 ...
##  $ Pclass  : int  3 1 3 1 3 3 1 3 3 2 ...
##  $ Sex     : chr  "male" "female" "female" "female" ...
##  $ Age     : num  22 38 26 35 35 NA 54 2 27 14 ...
##  $ SibSp   : int  1 1 0 1 0 0 0 3 0 1 ...
##  $ Parch   : int  0 0 0 0 0 0 0 1 2 0 ...
##  $ Fare    : num  7.25 71.28 7.92 53.1 8.05 ...
##  $ Embarked: chr  "S" "C" "S" "S" ...
```

Remove Missing Values

```r
titanic <- na.omit(titanic)
str(titanic)
```

```
## 'data.frame':    714 obs. of  8 variables:
##  $ Survived: int  0 1 1 1 0 0 0 1 1 1 ...
##  $ Pclass  : int  3 1 3 1 3 1 3 3 2 3 ...
##  $ Sex     : chr  "male" "female" "female" "female" ...
##  $ Age     : num  22 38 26 35 35 54 2 27 14 4 ...
##  $ SibSp   : int  1 1 0 1 0 0 3 0 1 1 ...
##  $ Parch   : int  0 0 0 0 0 0 1 2 0 1 ...
##  $ Fare    : num  7.25 71.28 7.92 53.1 8.05 ...
##  $ Embarked: chr  "S" "C" "S" "S" ...
##  - attr(*, "na.action")= 'omit' Named int [1:177] 6 18 20 27 29 30 32 33 37 43 ...
##   ..- attr(*, "names")= chr [1:177] "6" "18" "20" "27" ...
```

Factorize Column

```
titanic$Survived <- as.factor(titanic$Survived)
titanic$Pclass <- as.factor(titanic$Pclass)
```

```
str(titanic)
```

```
## 'data.frame':    714 obs. of  8 variables:
##  $ Survived: Factor w/ 2 levels "0","1": 1 2 2 2 1 1 1 2 2 2 ...
##  $ Pclass  : Factor w/ 3 levels "1","2","3": 3 1 3 1 3 1 3 3 2 3 ...
##  $ Sex     : chr  "male" "female" "female" "female" ...
##  $ Age     : num  22 38 26 35 35 54 2 27 14 4 ...
##  $ SibSp   : int  1 1 0 1 0 0 3 0 1 1 ...
##  $ Parch   : int  0 0 0 0 0 0 1 2 0 1 ...
##  $ Fare    : num  7.25 71.28 7.92 53.1 8.05 ...
##  $ Embarked: chr  "S" "C" "S" "S" ...
##  - attr(*, "na.action")= 'omit' Named int [1:177] 6 18 20 27 29 30 32 33 37 43 ...
##   ..- attr(*, "names")= chr [1:177] "6" "18" "20" "27" ...
```

Data Splitting

```
set.seed(123)
training.samples <- titanic$Survived %>% createDataPartition(p=0.75, list=FALSE)
train.data <- titanic[training.samples,]
test.data <- titanic[-training.samples,]
```

Prune CART

```
set.seed(123)

model_CART_c <- train(
  Survived ~.,
  data = train.data,
  method = "rpart",
  trControl = trainControl("cv", number = 10)
)
```

Access the best tune

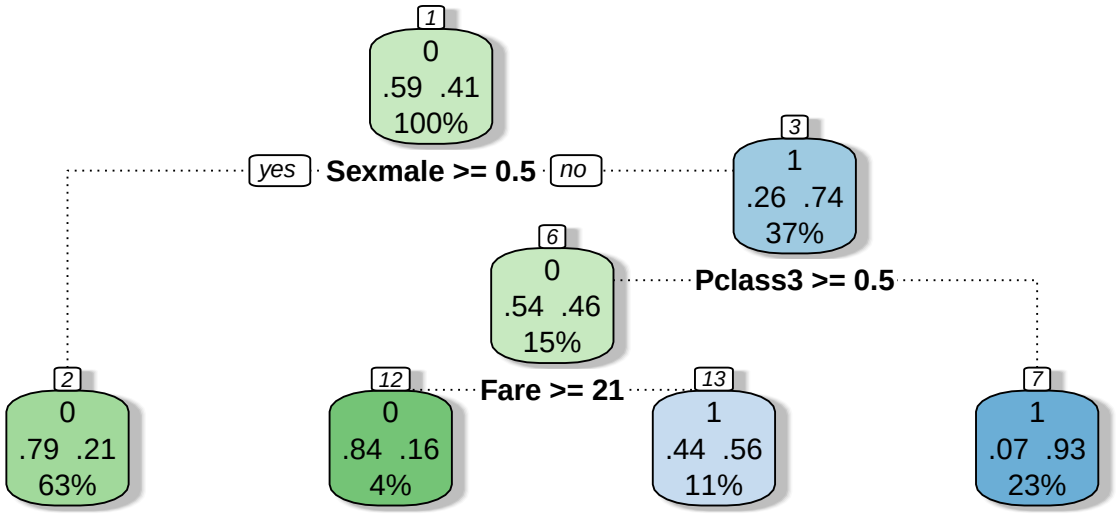
```
model_CART_c$bestTune
```

```
##           cp
## 1 0.01376147
```

The best CP value is 0.01376147

Plot the best model

```
fancyRpartPlot(model_CART_c$finalModel)
```



Prediction

```
pred_CART_c <- predict(
  model_CART_c,
  test.data
)
```

Confusion Matrix

```
cm1 <- confusionMatrix(
  factor(pred_CART_c),
  factor(test.data$Survived),
  positive = "1"
)
cm1
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 97 23
##          1  9 49
##
```

```
##                Accuracy : 0.8202
##                  95% CI : (0.7558, 0.8737)
##     No Information Rate : 0.5955
##     P-Value [Acc > NIR] : 1.056e-10
##
##                   Kappa : 0.6148
##
##  Mcnemar's Test P-Value : 0.02156
##
##             Sensitivity : 0.6806
##             Specificity : 0.9151
##          Pos Pred Value : 0.8448
##          Neg Pred Value : 0.8083
##              Prevalence : 0.4045
##          Detection Rate : 0.2753
##    Detection Prevalence : 0.3258
##       Balanced Accuracy : 0.7978
##
##        'Positive' Class : 1
##
```

```
cm1$byClass["Sensitivity"]
```

Sensitivity

```
## Sensitivity
##   0.6805556
```

```
cm1$byClass["Specificity"]
```

Specificity

```
## Specificity
##   0.9150943
```

```
cm1$overall["Accuracy"]
```

Overall Accuracy

```
##  Accuracy
## 0.8202247
```

Best Random Forest

```r
set.seed(123)
# 10-fold CV
control <- trainControl(
  method = "cv",
  number = 10
  )

# train the rf using 10-fold CV
model_rf_c <- train(
  Survived ~ .,
  data = train.data,
  method = "rf",
  trControl = control,
  importance = TRUE
  )

print(model_rf_c)
```

```
## Random Forest
##
## 536 samples
##   7 predictor
##   2 classes: '0', '1'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 482, 482, 482, 482, 483, 483, ...
## Resampling results across tuning parameters:
##
##   mtry  Accuracy   Kappa
##    2    0.7948987  0.5573048
##    6    0.7816562  0.5410341
##   10    0.7517470  0.4835361
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 2.
```

Access bestTune

```r
model_rf_c$bestTune
```

```
##   mtry
## 1    2
```

The best mtry is 2

Plot importance measures

```r
varImpPlot(model_rf_c$finalModel)
```

Prediction

```r
pred_rf_c <- predict(
  model_rf_c,
  newdata = test.data
)
```

Confusion Matrix

```r
cm2 <- confusionMatrix(
  pred_rf_c,
  test.data$Survived,
  positive = "1"
)
cm2
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 91 20
```

```
##          1 15 52
##
##                Accuracy : 0.8034
##                  95% CI : (0.7373, 0.8591)
##     No Information Rate : 0.5955
##     P-Value [Acc > NIR] : 2.711e-09
##
##                   Kappa : 0.5873
##
##  Mcnemar's Test P-Value : 0.499
##
##             Sensitivity : 0.7222
##             Specificity : 0.8585
##          Pos Pred Value : 0.7761
##          Neg Pred Value : 0.8198
##              Prevalence : 0.4045
##          Detection Rate : 0.2921
##    Detection Prevalence : 0.3764
##       Balanced Accuracy : 0.7904
##
##        'Positive' Class : 1
##
```

```
cm2$byClass["Sensitivity"]
```

Sensitivity

```
## Sensitivity
##   0.7222222
```

```
cm2$byClass["Specificity"]
```

Specificity

```
## Specificity
##   0.8584906
```

```
cm2$overall["Accuracy"]
```

Overall Accuracy

```
##  Accuracy
## 0.8033708
```

Which is better?

In terms of sensitivity, Random Forest is better

In terms of specificity, CART is beter

In terms of overall accuracy, CART is better

The reason behind the difference in performance:

- Generally Random Forest relies on sub-seting random features to create multiple CART model. This dataset only has 7 features, so the Random Forest doesn't get much exposure to more diverse data.